NEXA Digital School

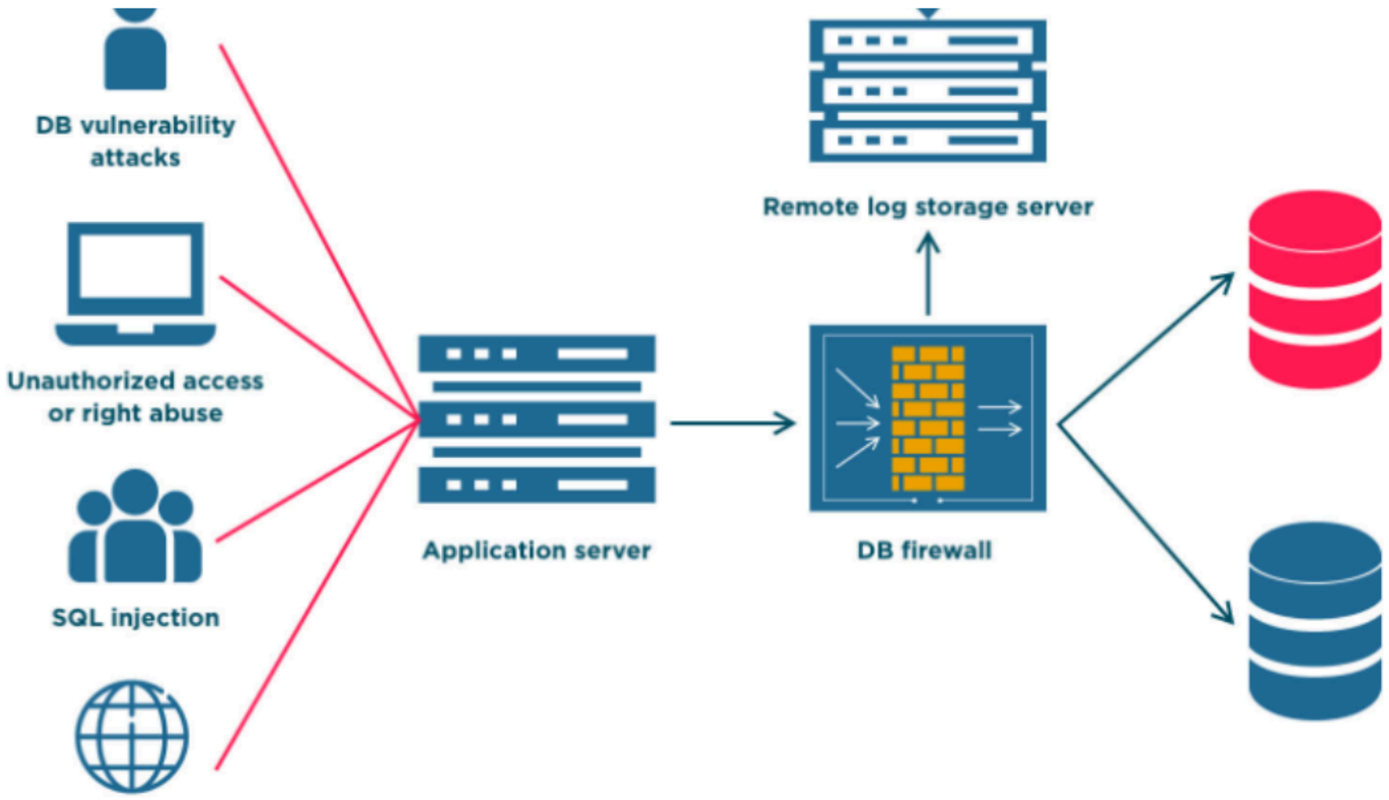
29 AVRIL 2026

MASTÈRE DATA & IA

CHEF DE PROJET

Sécurité des bases de données

DB Security — Accès & Attaques



FORMATEUR : DIALLO ALIMOU

Objectifs Pédagogiques

- Identifier les principales menaces (exemple : **injection SQL**)
- Comprendre et tester une **injection SQL** simple
- Créer et gérer des utilisateurs sous **PostgreSQL**
- Attribuer des droits (**GRANT / REVOKE**)
- Mettre en place des rôles **(RBAC**) et appliquer le moindre privilège

Qu'est-ce que la sécurité de la base de données

La sécurité des bases de données combine **technologies**, **politiques** et **processus** pour **protéger les données stockées dans les bases de données contre les accès non autorisés**, **les violations ou la corruption**. Elle couvre la gestion des **accès**, l'**audit**, le **chiffrement**, la **surveillance** et la **récupération** — garantissant la **confidentialité**, l'**intégrité** et la **disponibilité**. Plutôt que de se concentrer uniquement sur les défenses réseau ou périmétriques, une sécurité efficace des bases de données protège les données elles-mêmes.

Pourquoi sécuriser une base ?

Données sensibles :

- **Clients** : identité, emails, informations personnelles
- **Mots de passe** : accès aux comptes utilisateurs
- **Transactions** : opérations financières, historiques

Risques :

- **Vol de donnée**s : revente, fraude, usurpation d'identité
- **Modification frauduleuse** : altération des soldes, fausses transactions
- **Indisponibilité** : service bloqué, perte d'accès aux données

Une base non sécurisée = risque technique + impact financier et légal

TRIADE CIA

Confidentialité

- Protéger les données contre les accès non autorisés
  - Accès limité aux utilisateurs autorisés
  - Gestion des droits (GRANT / REVOKE)
  - Protection des informations sensibles

Intégrité

- Garantir que les données sont correctes et non altérées
  - Empêcher les modifications frauduleuses
  - Assurer la cohérence des données
  - Utiliser contraintes et contrôles

TRIADE CIA

Disponibilité

- Assurer l’accès aux données à tout moment
  - Systèmes disponibles et fonctionnels
  - Sauvegardes régulières
  - Protection contre pannes et attaques

Objectif : Protéger les données sans empêcher leur utilisation

TRIADE CIA

Disponibilité

- Assurer l’accès aux données à tout moment
  - Systèmes disponibles et fonctionnels
  - Sauvegardes régulières
  - Protection contre pannes et attaques

Objectif : Protéger les données sans empêcher leur utilisation

MENACES PRINCIPALES

Menaces principales

- **Injection SQL**
  - Exploite des requêtes vulnérables pour accéder ou manipuler les données
- **Mauvaise gestion des droits**
  - Trop d’autorisations données aux utilisateurs → accès non justifié
- **Comptes trop puissants**
  - Utilisation de comptes admin / superuser → risque critique en cas de compromission
- **Attaques internes**
  - Employés ou utilisateurs abusant de leurs accès

MENACES PRINCIPALES

Voir le document pdf sur les Exemples de Scénarios de Menaces en Matière de Sécurité dans les SGBD

INJECTION SQL (PRINCIPE)

Une **injection SQL (SQLi)** est un type d'attaque dans laquelle les cybercriminels tentent d'exploiter les **vulnérabilités du code d'une application** en insérant une **requête SQ**L dans des champs de saisie ou de formulaire réguliers, tels qu'un nom d'utilisateur ou un mot de passe. L'instruction SQL est ensuite transmise à la base de données SQL sous-jacente de l'application.

Requête normale :

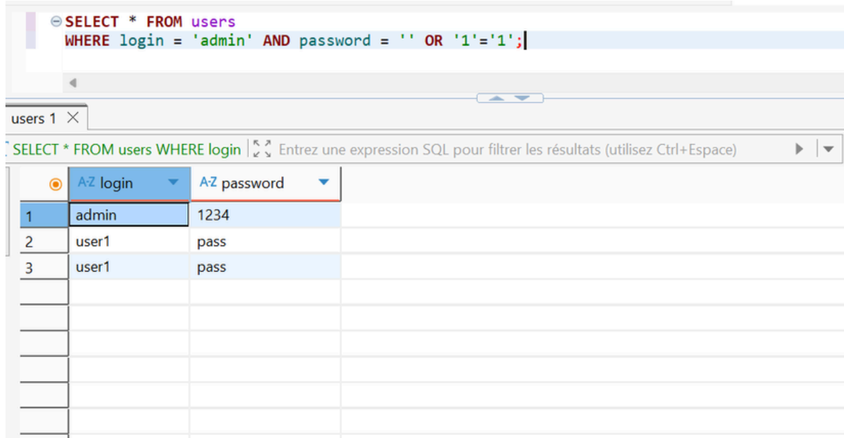
SELECT * FROM users WHERE login='admin' AND password='123';

Injection :

' OR '1'='1

- Résultat : accès sans mot de passe

SELECT * FROM users WHERE login = '' OR '1'='1';



INJECTION SQL (PRINCIPE)

1. Requête normale (connexion valide)

```
SELECT * FROM users
WHERE login = 'admin' AND password = '1234';
```

2. Injection SQL (attaque)

```
SELECT * FROM users WHERE login = '' OR '1'='1';
```

Résultat :

Tous les utilisateurs sont retournés

Authentification contournée

Requête sécurisée (principe)

- Utiliser des requêtes préparées

```
SELECT * FROM users
WHERE login = ? AND password = ?;
```

AUTHENTIFICATION VS AUTORISATION

Authentification

Qui es-tu ?

- Vérifie l'identité de l'utilisateur
- Exemple : login / mot de passe
- Peut inclure : MFA (code, SMS, etc.)

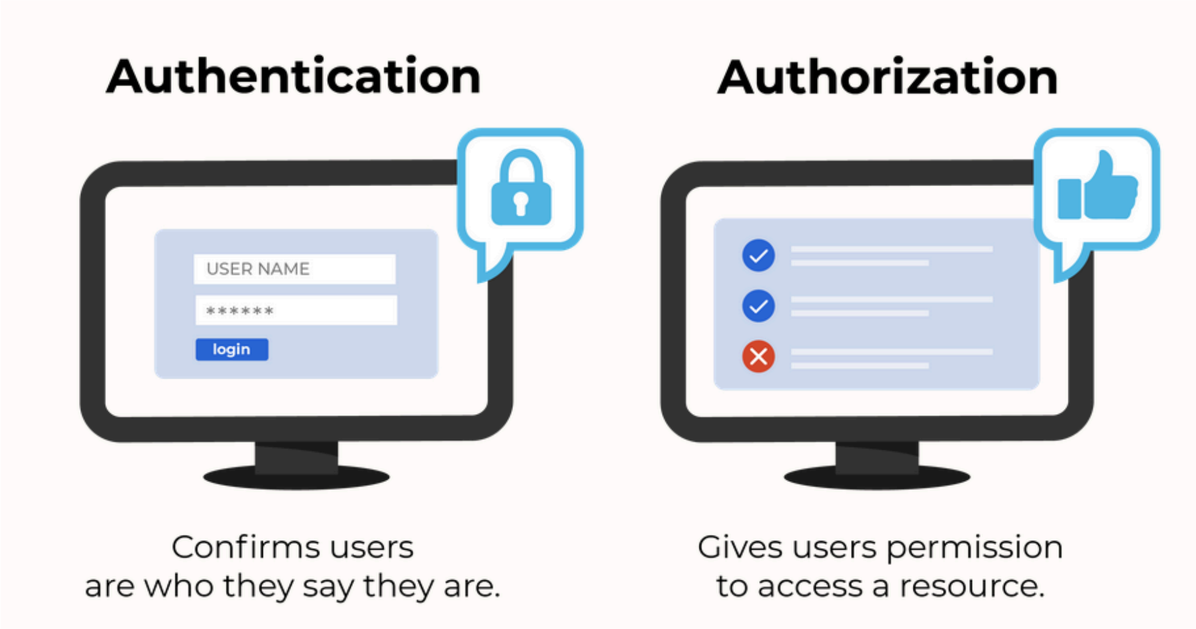
Résultat : utilisateur reconnu

Autorisation

Que peux-tu faire ?

- Définit les actions autorisées
- Exemple :
  - lire des données
  - modifier
  - supprimer

Résultat : accès limité selon le rôle




```
CREATE USER 'user1' IDENTIFIED BY '1234';

GRANT SELECT ON bank.* TO 'user1';
REVOKE INSERT ON bank.* FROM 'user1';
```

PRINCIPE DU MOINDRE PRIVILÈGE

Donner à chaque utilisateur uniquement les droits nécessaires pour accomplir sa tâche .

Mauvaise pratique

- Donner tous les droits (**admin** / **superuser**)
- Risque élevé en cas d’erreur ou d’attaque
- Peut entraîner : suppression ou modification massive

Bonne pratique

- Limiter les permissions au strict nécessaire
- Séparer les rôles (lecture, écriture, admin)
- Réduire la surface d’attaque

Moins un utilisateur a de droits, moins le risque est grand

PRINCIPE DU MOINDRE PRIVILÈGE

Donner à chaque utilisateur uniquement les droits nécessaires pour accomplir sa tâche .

Mauvaise pratique

- Donner tous les droits (**admin** / **superuser**)
- Risque élevé en cas d’erreur ou d’attaque
- Peut entraîner : suppression ou modification massive

Bonne pratique

- Limiter les permissions au strict nécessaire
- Séparer les rôles (lecture, écriture, admin)
- Réduire la surface d’attaque

Moins un utilisateur a de droits, moins le risque est grand

RBAC (ROLE-BASED ACCESS CONTROL)

Donner à chaque utilisateur uniquement les droits nécessaires pour accomplir sa tâche .

Mauvaise pratique

- Donner tous les droits (**admin** / **superuser**)
- Risque élevé en cas d’erreur ou d’attaque
- Peut entraîner : suppression ou modification massive

Bonne pratique

- Limiter les permissions au strict nécessaire
- Séparer les rôles (lecture, écriture, admin)
- Réduire la surface d’attaque

Moins un utilisateur a de droits, moins le risque est grand

RBAC (ROLE-BASED ACCESS CONTROL)

Principe :

- Attribuer des rôles aux utilisateurs
- Ne pas gérer les droits utilisateur par utilisateur

Fonctionnement

- On crée des rôles (ex : admin, conseiller, analyste)
- On attribue des droits aux rôles
- On affecte les utilisateurs à ces rôles
    - Simplifier la gestion des accès
    - Éviter les erreurs de configuration
    - Standardiser les permissions

```
CREATE ROLE conseiller;
GRANT SELECT ON clients TO conseiller;
```

TP1 — SAFEBANK DB

TP1 — SafeBank DB (Sécurité des...

1. Question de compréhension

1. Question de compréhension

Contexte

Vous travaillez sur une application bancaire connectée à une base de données. Cette application gère des informations sensibles telles que les données clients, les comptes et les agences. Ces données doivent être protégées contre les accès non autorisés.

Objectif

Sécuriser une base de données bancaire en mettant en place une gestion correcte des utilisateurs, des droits et des rôles.

Travail demandé

1. Création de la base

Créer une base de données contenant les tables suivantes : clients, comptes.

2. Gestion des utilisateurs

Créer deux utilisateurs :
- admin (gestion technique)
- app_user (utilisateur de l'application)

3. Sécurité des accès

Mettre en place les droits d'accès en utilisant GRANT et REVOKE.